

Modelo de datos de Regenta

Traducción del documento de arquitectura y del `CLAUDE.md` a un modelo entidad-relación ejecutable: *147 tablas repartidas en 15 esquemas*, uno por microservicio, con aislamiento multi-tenant por `negocio_id` y los tres patrones operativos resueltos por configuración, no por código.

147 TABLAS	15 ESQUEMAS	3 PATRONES	0 FK ENTRE SERVICIOS	14 HALLAZGOS
---------------	----------------	---------------	----------------------------	-----------------

Proyecto Regenta — app modular Android + Web	Verificado contra PostgreSQL 16 · los 16 scripts DDL se ejecutan sin errores
Entregables modelo-datos/sql · modelo-datos/jpa · modelo-datos/drawio	

Contenido

01	Qué se está proponiendo, en una lectura	Los tres patrones y qué debe garantizar cada uno
02	Siete reglas que se aplican a las 147 tablas	UUID, negocio_id, RLS, cero FK entre servicios, Outbox, tipos, snapshots
03	Mapa de contextos	Los 15 microservicios: qué es REST y qué es evento
04	Los tres patrones, lado a lado	La misma tabla sirviendo a ferretería, papelería y droguería
05	Modelo entidad-relación por contexto	Diagramas y diccionario de datos de los 15 esquemas
06	Modelo de dominio: agregados e invariantes	Qué protege el código y qué protege PostgreSQL
07	Mapa de eventos	Quién publica qué y quién lo consume
08	Inventario completo de tablas	Las 147 tablas por esquema
09	Catorce hallazgos del análisis	Dónde el diseño propuesto no aguanta lo que los planes prometen
10	Archivos entregados	Scripts, entidades y cómo levantarlo en local

01 Qué se está proponiendo, en una lectura

Regenta es un SaaS multi-tenant donde **un solo backend sirve a todos los negocios clientes**. Vender a un cliente nuevo es insertar una fila en `negocios` y crear su primer admin — no se despliega infraestructura.

La decisión de diseño más fuerte del documento es la **generalización por patrones operativos**: ferretería, papelería, droguería, veterinaria y vidriería no son verticales distintos, son el mismo patrón *Venta directa* con categorías y atributos distintos. Solo tres patrones existen, y lo que los separa no es el rubro sino **la forma de la transacción**:

- **VENTA DIRECTA** — la transacción se abre y se cierra en un instante y consume stock.
- **RESERVA** — la transacción ocupa un **intervalo de tiempo** sobre un recurso que no se consume.
- **COMANDA** — la transacción queda **abierta** y acumula líneas durante minutos u horas antes de cerrarse.

Eso tiene una consecuencia directa en el modelo de datos, y es la tesis de este documento: **los tres patrones comparten el Core completo y comparten la forma general (un módulo de catálogo del que depende un módulo de transacción), pero cada uno necesita una estructura de datos y una garantía de integridad que los otros dos no necesitan**. Ese es el trabajo que este modelo hace explícito.

PATRÓN	CATÁLOGO	TRANSACCIÓN	LO QUE EL MODELO DEBE GARANTIZAR	DÓNDE
VENTA DIRECTA	productos	ventas	Que el stock no se descuente dos veces ni quede en negativo cuando dos cajas venden el mismo producto	Libro mayor append-only + saga + <code>idempotency_key</code>
RESERVA	recursos	reservas	Que dos reservas del mismo recurso nunca se solapen en el tiempo	Constraint <code>EXCLUDE</code> GiST
COMANDA	items_menu	comandas	Que cada línea tenga su propio ciclo de vida y que al cerrar se descuenten los insumos	Estado por línea + tabla <code>recetas</code>

LA DISTINCIÓN QUE ORDENA TODO EL MODELO

Configuración y código no son lo mismo. Un tipo de negocio nuevo se resuelve con **filas** (`categorias`, `atributos_categoria`, el JSONB `atributos`). Un patrón nuevo se resuelve con **tablas**. Si al analizar un cliente nuevo la pregunta es «¿cómo se llaman sus productos?», es configuración. Si es «¿su venta ocupa tiempo, o queda abierta?», es un patrón.

02 Siete reglas que se aplican a las 147 tablas

Antes del detalle por contexto, las decisiones transversales. Están en `00-convenciones.sql` y no se negocian tabla por tabla.

1 • Toda PK es UUID, nunca `BIGSERIAL`

La app es offline-first: un vendedor sin señal registra una venta en el dispositivo y `workmanager` la sube después. Si el id lo asignara el servidor, esa venta no tendría cómo referenciarse localmente ni cómo deduplicarse al sincronizar. El cliente genera el UUID (idealmente v7, ordenable en el tiempo para no fragmentar los índices B-tree) y el servidor lo respeta.

2 • `negocio_id` en toda tabla de negocio, y siempre primero en el índice

Un índice compuesto que no empiece por `negocio_id` es un bug de rendimiento en multi-tenant: obliga a escanear filas de todos los negocios para filtrar después. Las únicas tablas sin `negocio_id` son catálogos globales del SaaS (`planes`, `modulos`, `permisos`, `dim_fecha`) y tablas puente que heredan el tenant de su padre.

3 • El aislamiento se defiende en tres capas independientes

El filtro de la aplicación es la primera; Row-Level Security en Postgres es la red por si un `WHERE` se olvida; el claim del JWT validado en el Gateway es la tercera. Ninguna basta sola.

```
-- Cada tabla de negocio:
ALTER TABLE productos ENABLE ROW LEVEL SECURITY;
ALTER TABLE productos FORCE ROW LEVEL SECURITY; -- aplica también al owner
CREATE POLICY tenant_isolation ON productos
  USING      (negocio_id = app_negocio_actual())
  WITH CHECK (negocio_id = app_negocio_actual());

-- Y al inicio de CADA transacción, desde el backend:
SET LOCAL app.negocio_id = '<claim negocio_id del JWT>';
```

EL ERROR QUE HAY QUE EVITAR AQUÍ

Tiene que ser `SET LOCAL`, no `SET`. HikariCP reutiliza conexiones entre peticiones: un `SET` normal deja el `negocio_id` del tenant anterior pegado a la conexión y el siguiente request ve datos ajenos. Va en un interceptor/AOP atado a la transacción, nunca en la configuración del pool.

4 • Cero claves foráneas entre esquemas de servicios distintos

Cada microservicio tiene su base. Una FK física a la tabla de otro servicio es imposible en producción y acopla los despliegues. En su lugar, tres mecanismos, y el DDL marca cada columna con `-- [ref lógica]`:

- **Referencia lógica** — se guarda el UUID sin `REFERENCES`. La integridad la garantiza el servicio dueño (`ventas.cliente_id`).
- **Snapshot desnormalizado** — se copia lo que debe quedar inmutable. Una factura emitida no puede cambiar porque alguien editó el nombre del cliente después (`facturas.cliente_snapshot JSONB`).
- **Réplica por evento** — una copia de solo lectura alimentada por RabbitMQ (`reportes.dim_producto`).

5 • Outbox e Inbox en todo servicio que publique o consuma

Sin **Outbox** no hay atomicidad entre «guardar la venta» y «publicar `venta_completada`»: si RabbitMQ está caído después del commit, el evento se pierde y Facturación nunca emite. El evento se *inserta* en la misma transacción que la venta y un publicador aparte lo envía.

Sin **Inbox** no hay idempotencia: RabbitMQ entrega *at-least-once*, así que el mismo `venta_completada` puede llegar dos veces y facturar dos veces.

6 · Dinero en `NUMERIC`, tiempo en `TIMESTAMPTZ`, enums como `VARCHAR + CHECK`

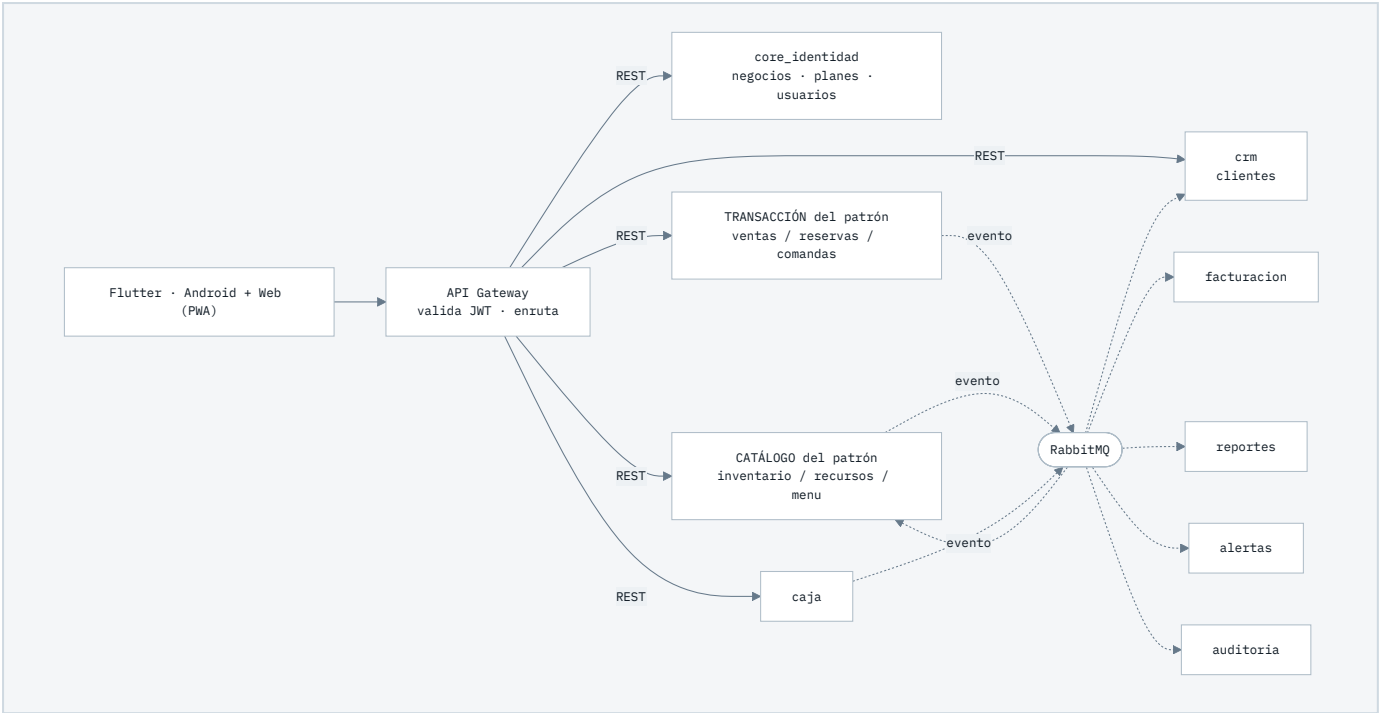
`NUMERIC(18,4)` para cálculo interno y `(14,2)` para totales presentables; nunca `float`. `TIMESTAMPTZ` siempre, porque los negocios pueden estar en zonas distintas. Los enums van como `CHECK` y no como tipo `ENUM` de Postgres: agregar un valor a un `ENUM` pide DDL con lock, un `CHECK` se cambia sin bloquear escrituras largas.

7 · Precios e impuestos se congelan en la línea, no se leen del catálogo

`venta_lineas` guarda `sku_snapshot`, `nombre_snapshot`, `precio_unitario`, `impuesto_pct` y `costo_unitario_snapshot`. Reimprimir una venta de hace tres meses debe mostrar el precio de entonces, y el margen del reporte se calcula contra el costo que había ese día.

03 Mapa de contextos

Quince esquemas. El Gateway habla REST síncrono con lo que bloquea al usuario; todo lo demás viaja por RabbitMQ. Los cuatro servicios de la derecha no consultan a nadie: viven de eventos.



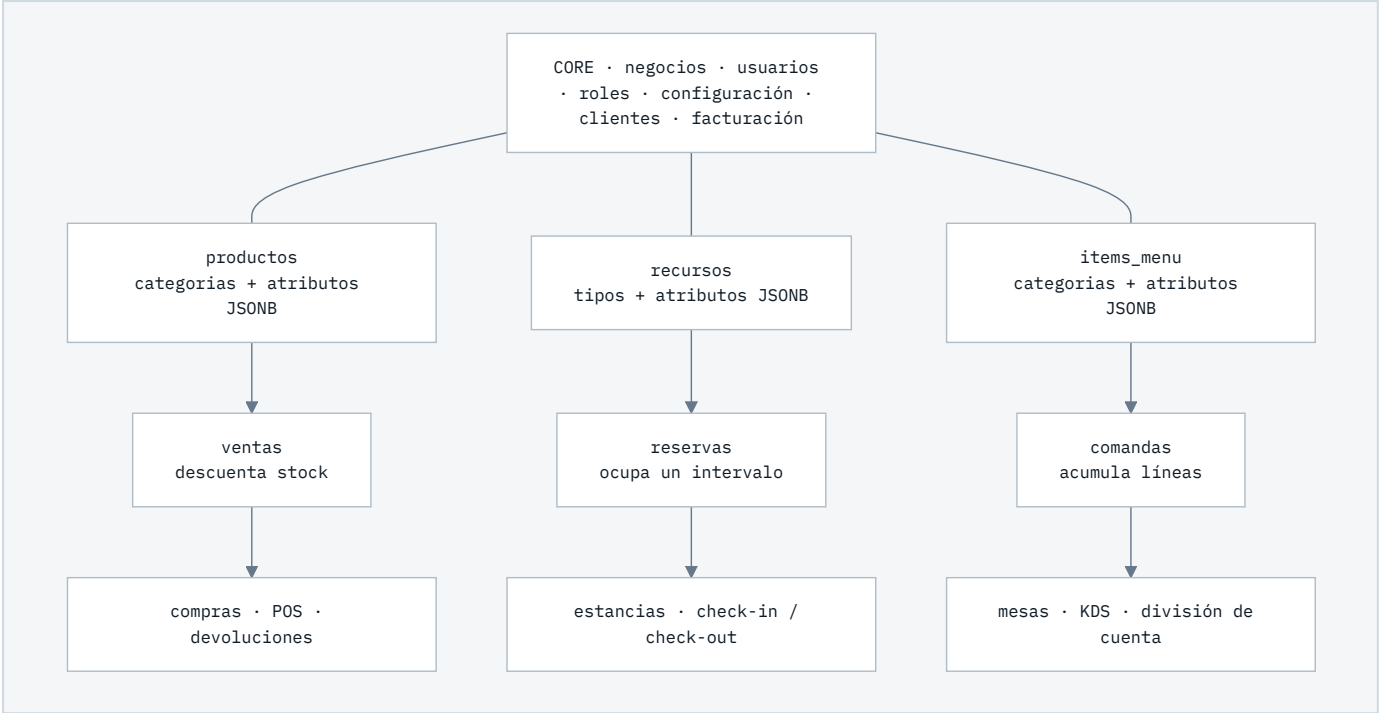
Línea sólida = REST síncrono, el usuario espera respuesta. Línea punteada = evento asíncrono; el emisor no sabe quién consume.

ESQUEMA	SERVICIO	PATRÓN	ROL	TABLAS	PLAN MÍNIMO
core_identicidad	servicio-usuarios	CORE	Tenant, planes, módulos, usuarios, roles, sucursales, impuestos	21	Todos
crm	servicio-clientes propuesto	CORE	Clientes, cartera, interacciones	7	Profesional
inventario	servicio-inventario	VENTA DIRECTA	Catálogo, stock, lotes, movimientos	21	Básico
ventas	servicio-ventas	VENTA DIRECTA	Transacción, pagos, devoluciones, saga	8	Básico
compras	servicio-compras	VENTA DIRECTA	Proveedores, órdenes, recepciones, CxP	8	Profesional
recursos	servicio-recursos	RESERVA	Tipos, recursos, tarifas, bloqueos	8	Básico
reservas	servicio-reservas	RESERVA	Reservas, estancias, consumos, pagos	9	Básico
menu	servicio-menu	COMANDA	Cartas, ítems, modificadores, recetas	10	Básico
mesas	servicio-mesas	COMANDA	Zonas, mesas, sesiones	5	Básico
comandas	servicio-comandas	COMANDA	Comandas, líneas, cuentas, KDS	9	Básico
facturacion	servicio-facturacion	CORE	Resoluciones DIAN, facturas, transmisión	7	Profesional
caja	servicio-caja propuesto	CORE	Cajas, arqueo, movimientos, turnos	5	Empresarial
alertas	servicio-alertas propuesto	CORE	Reglas, alertas, entregas, tokens FCM	6	Profesional
reportes	servicio-reportes	CORE	Estrella: hechos + dimensiones + agregados	16	Profesional
auditoria	servicio-auditoria propuesto	CORE	Bitácora, cola de sync, conflictos	6	Empresarial

Los marcados **propuesto** son módulos que sí están en la tabla de planes del documento pero no tienen servicio en el scaffold de repos del `CLAUDE.md`. Ver hallazgos **02** y **03**.

04 Los tres patrones, lado a lado

La forma es la misma en los tres: un módulo de **catálogo** del que depende un módulo de **transacción**, ambos colgando del Core. Lo que cambia es qué garantiza cada uno.



Ferretería, papelería, droguería, veterinaria y vidriería son todas la columna izquierda. Lo único que las diferencia son las filas de `categorias` y `atributos_categoria`.

La misma tabla, tres negocios distintos

	FERRETERÍA	PAPELERÍA	DROGUERÍA
categorias	Herramientas, Tornillería, Pintura	Útiles escolares, Oficina	Medicamentos, Cuidado personal
atributos_categoria	unidad_medida (metro, kilo) — obligatorio	—	lote, fecha_vencimiento, registro_sanitario — obligatorios
productos.atributos	{ "unidad_medida": "metro" }	{ }	{ "registro_sanitario": "INVIMA-2024-M-0123" }
productos.maneja_lotes	false	false	true + perecedero
Regla especial	—	—	Alertar vencimientos → una fila en reglas_alerta, no un módulo
Código nuevo	Ninguno en los tres casos		

POR QUÉ EL JSONB NO ES UNA EXCUSA PARA NO MODELAR

`atributos_categoria` es el contrato que valida el JSONB. El servicio comprueba el `atributos` de cada producto contra las filas de esa tabla antes de persistir; Postgres no valida la forma del JSON. Y si un atributo se filtra en *todas* las consultas (el vencimiento en una droguería), se promueve a columna real o a índice de expresión — el GIN genérico no rinde como un B-tree dedicado.

05

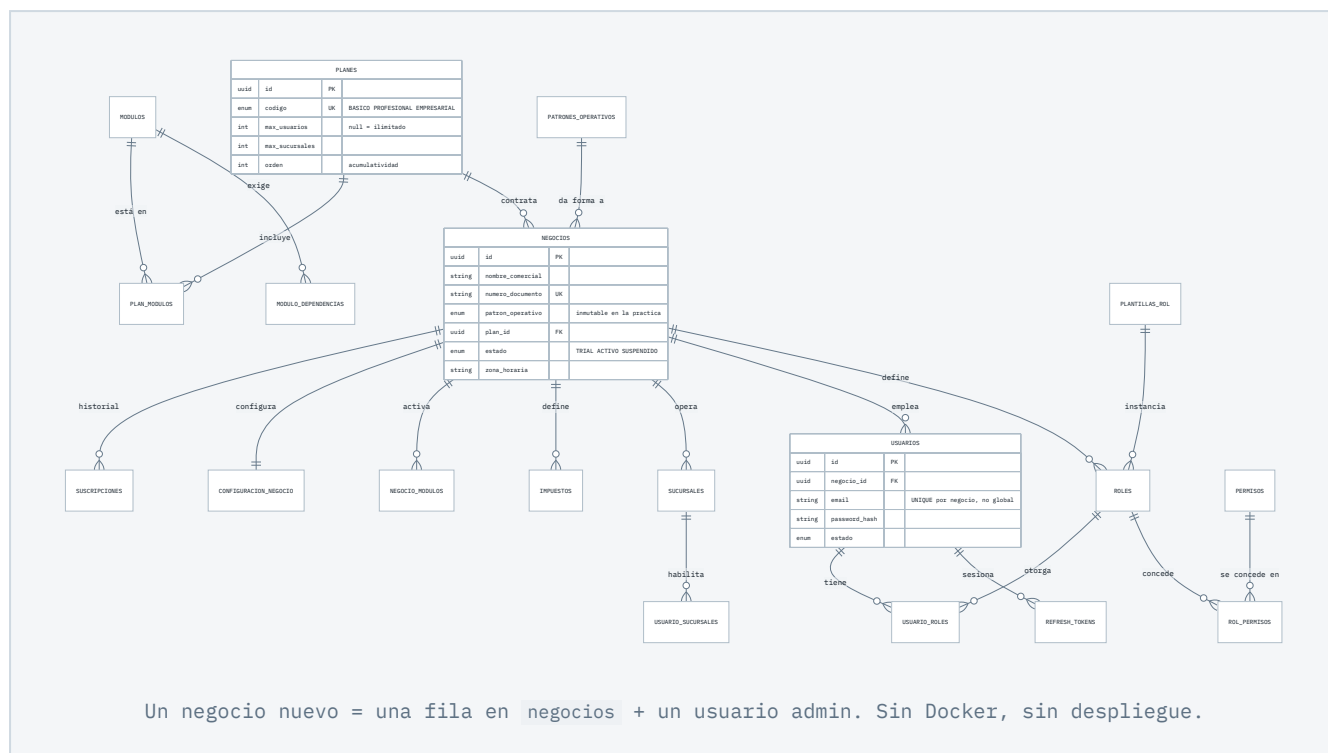
CÓMO USAR ESTA SECCIÓN

Cada diagrama va a página completa, y los que ganan con ello, girados para aprovechar el ancho. Aun así, un esquema de veintiuna entidades en una hoja A4 es un mapa, no una lupa: para leer columna por columna está `modelo-datos/drawio/`, con el mismo modelo en draw.io —una página por microservicio, con zoom—. Los diccionarios de datos de las tablas críticas sí están aquí, a tamaño de lectura.

CORE **core_identidad**

21 tablas · servicio-usuarios

El único servicio que conoce la tabla `negocios`. Los demás reciben `negocio_id`, `plan` y `patrón` como claims del JWT, no consultando esta tabla en cada request.



Diccionario · **negocios** y control de plan

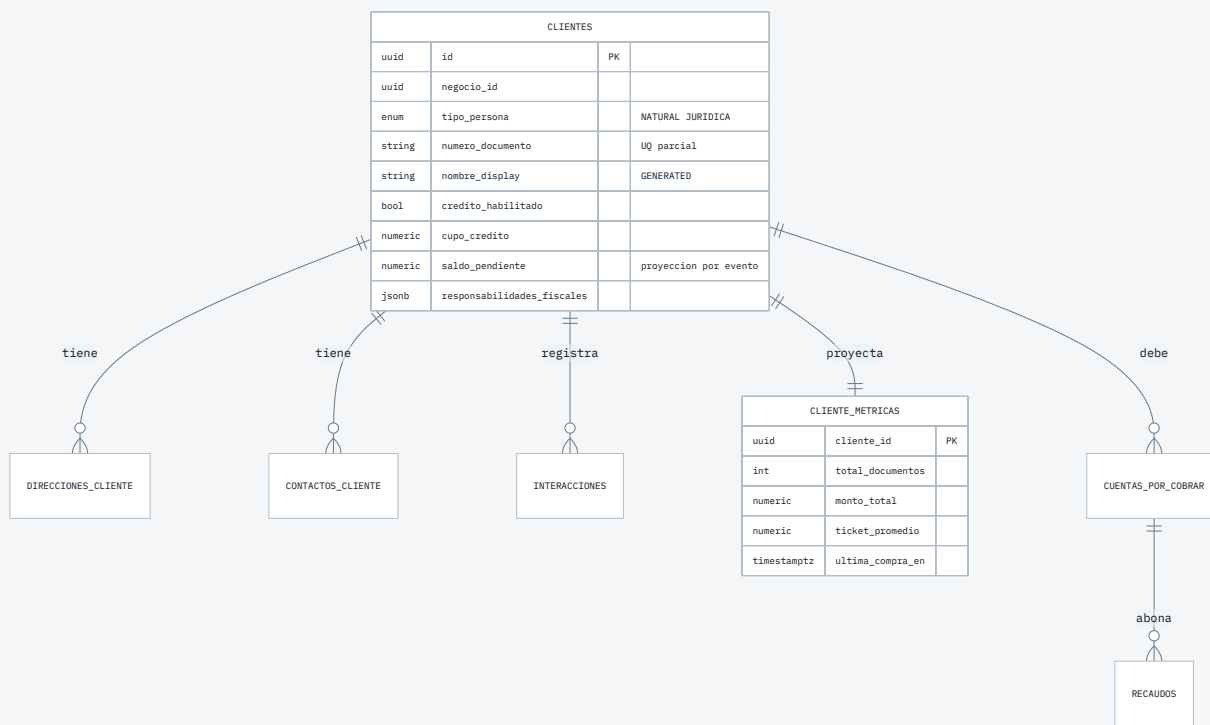
COLUMNA	TIPO	NOTAS
id PK	uuid	El tenant. Todo el sistema gira alrededor de este valor.
patron_operativo FK	varchar(30)	VENTA_DIRECTA · RESERVA · COMANDA . En la práctica inmutable: cambiarlo tras operar exige migrar datos entre modelos distintos. El servicio lo bloquea si ya hay transacciones.
plan_id FK	uuid	Plan vigente. El historial va en <code>suscripciones</code> (una vigente por negocio, garantizado por índice parcial).
estado	varchar(20)	TRIAL · ACTIVO · SUSPENDIDO · CANCELADO . El Gateway rechaza si no es ACTIVO/TRIAL.
numero_documento UQ	varchar(30)	Único junto con <code>pais</code> y <code>tipo_documento</code> .
zona_horaria	varchar(50)	Necesaria para «ventas de hoy»: los <code>TIMESTAMP TZ</code> se agrupan en la zona del negocio, no la del servidor.

CÓMO SE HACE EFECTIVO EL PLAN

`modulos` + `modulo_dependencias` forman un grafo: nunca se activa `FACTURACION` sin `VENTAS`, ni `RESERVAS` sin `RECURSOS`. `plan_modulos` materializa la lista *completa* por plan (no el delta) para que validar sea una lectura, no un recorrido del grafo en cada request. `negocio_modulos` permite add-ons: el documento marca POS como «Empresarial / add-on», y eso es una fila con `origen = 'ADDON'`.

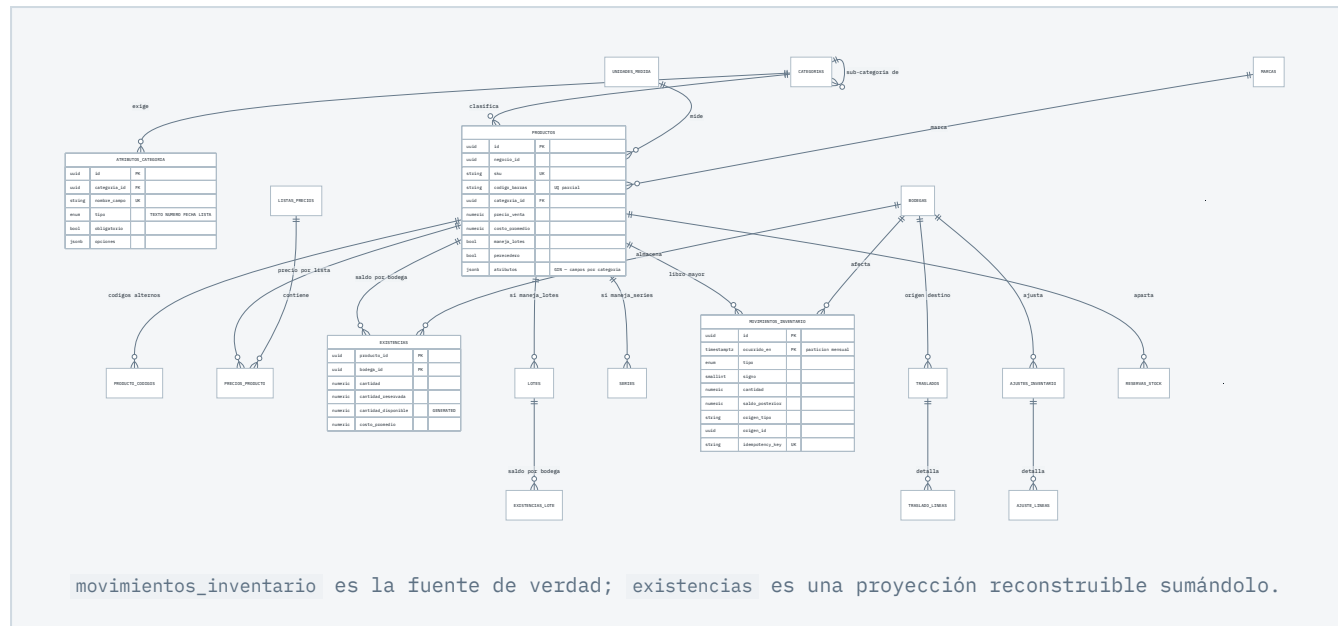
```
// La validación vive en el backend, no en el botón de Flutter
@RequiereModulo("FACTURACION")
@PostMapping("/facturas")
ResponseEntity<FacturaDto> emitir(...)
// 402 Payment Required si el plan no lo incluye – es un límite comercial,
// no un 403: el usuario tiene permiso, el negocio no tiene el módulo.
```

Consumido por Ventas, Reservas, Comandas y Facturación — los cuatro. Meterlo dentro de Ventas lo volvería una dependencia síncrona de todos, por eso se propone como servicio propio.



`cliente_metricas` no se calcula con un JOIN a Ventas — son servicios distintos con bases distintas. Se alimenta de `venta_completada`, `reserva_confirmada` y `pedido_completado`.

El esquema donde vive la generalización por configuración, y el que más se aparta del documento original.



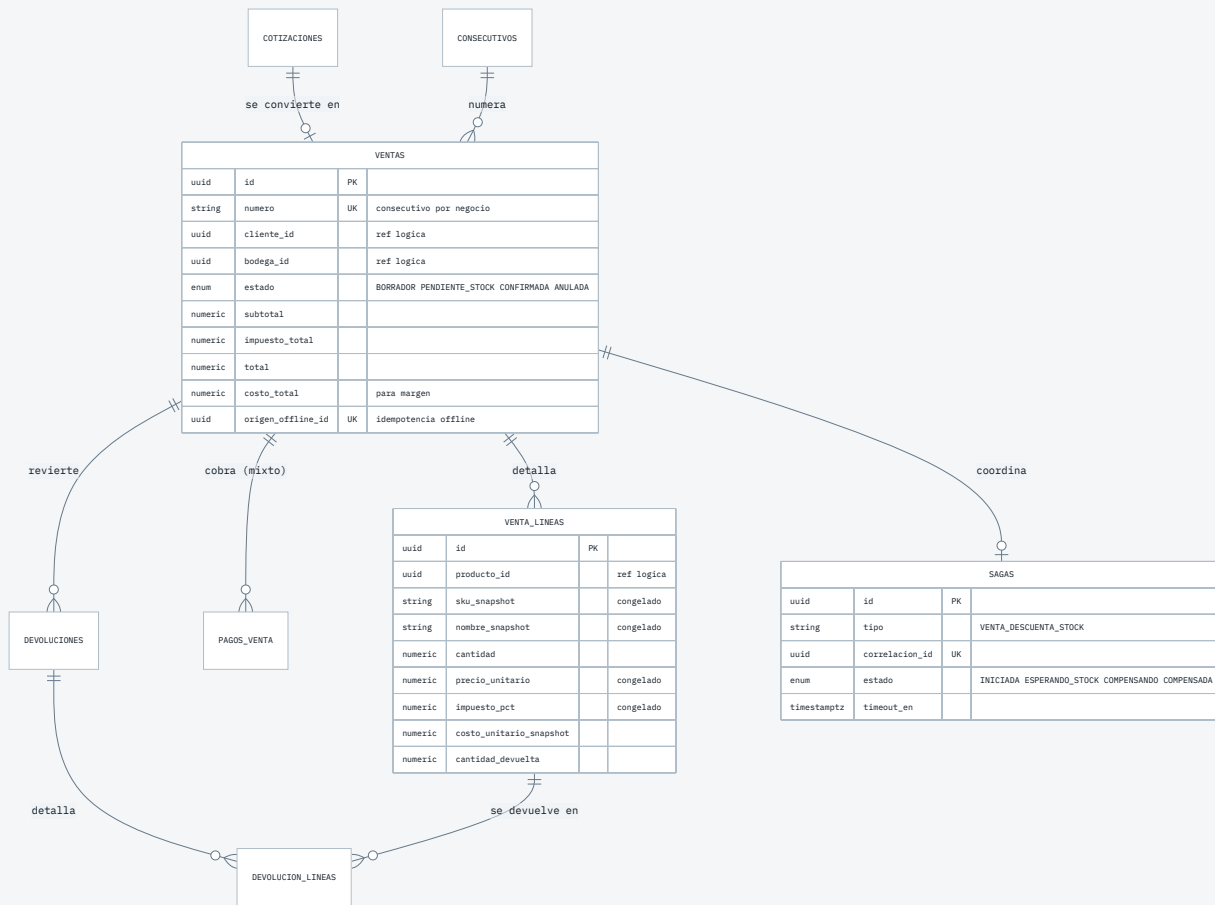
CORRECCIÓN SOBRE EL DOCUMENTO ORIGINAL

El PDF y el CLAUDE.md definen productos (... precio, **stock**, atributos JSONB), con el stock como columna del producto. Eso funciona solo mientras haya una bodega. El plan Empresarial vende **Multi-sucursal**, y POS necesita saber de qué bodega salió la mercancía. Mover el stock a *existencias* después obliga a reescribir todas las consultas de venta y todos los índices. Aquí se modela como (producto, bodega) desde el día 1. Ver hallazgo **01**.

Diccionario · movimientos_inventario (libro mayor, append-only)

COLUMNA	TIPO	NOTAS
id, ocurrido_en PK	uuid, timestampz	PK compuesta: la tabla se particiona por mes y Postgres exige que toda clave única incluya la columna de partición.
tipo	varchar(25)	11 valores: ENTRADA_COMPRA · SALIDA_VENTA · SALIDA_MERMA · RESERVA · LIBERACION_RESERVA ...
signo	smallint	-1 · 0 · +1. La cantidad siempre es positiva; el signo dice la dirección.
saldo_posterior	numeric(18,6)	Snapshot del saldo tras el movimiento. Permite auditar sin re-sumar todo el libro.
origen_tipo, origen_id	varchar, uuid ref lógica	Polimórfico: VENTA · COMPRA · AJUSTE · TRASLADO · COMANDA · RESERVA. Sirve a los tres patrones.
idempotency_key UQ	varchar(120)	La pieza crítica. RabbitMQ entrega <i>at-least-once</i> : si venta_completada llega dos veces, el segundo INSERT choca contra el UNIQUE y el stock no se descuenta dos veces.

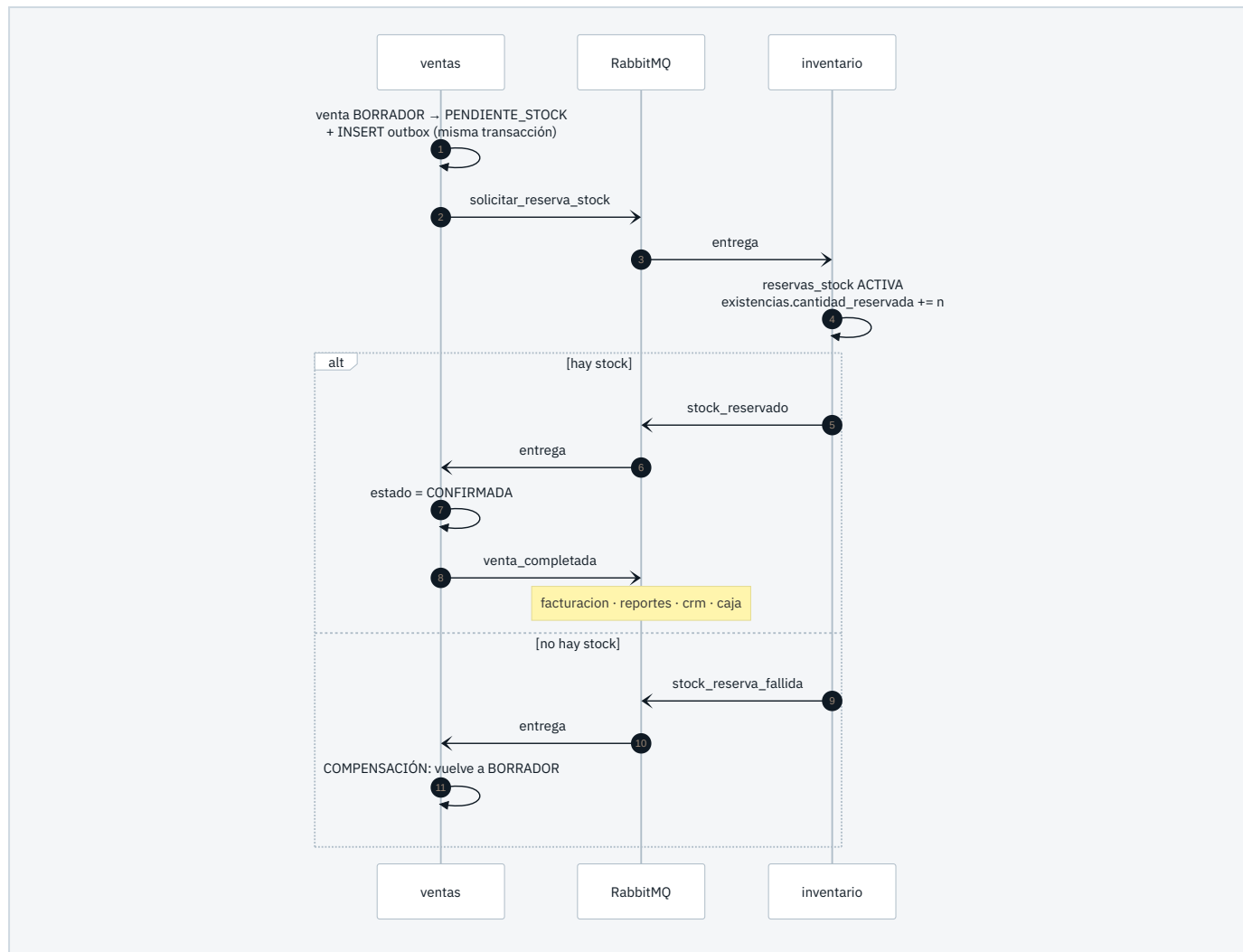
Nunca se actualiza ni se borra una fila de esta tabla. Un error se corrige con un movimiento contrario, igual que en contabilidad — así el histórico sigue siendo auditable y el saldo sigue siendo reconstruible.



El agregado es Venta + VentaLinea + PagoVenta . Todo lo demás es un UUID que vive en otro servicio.

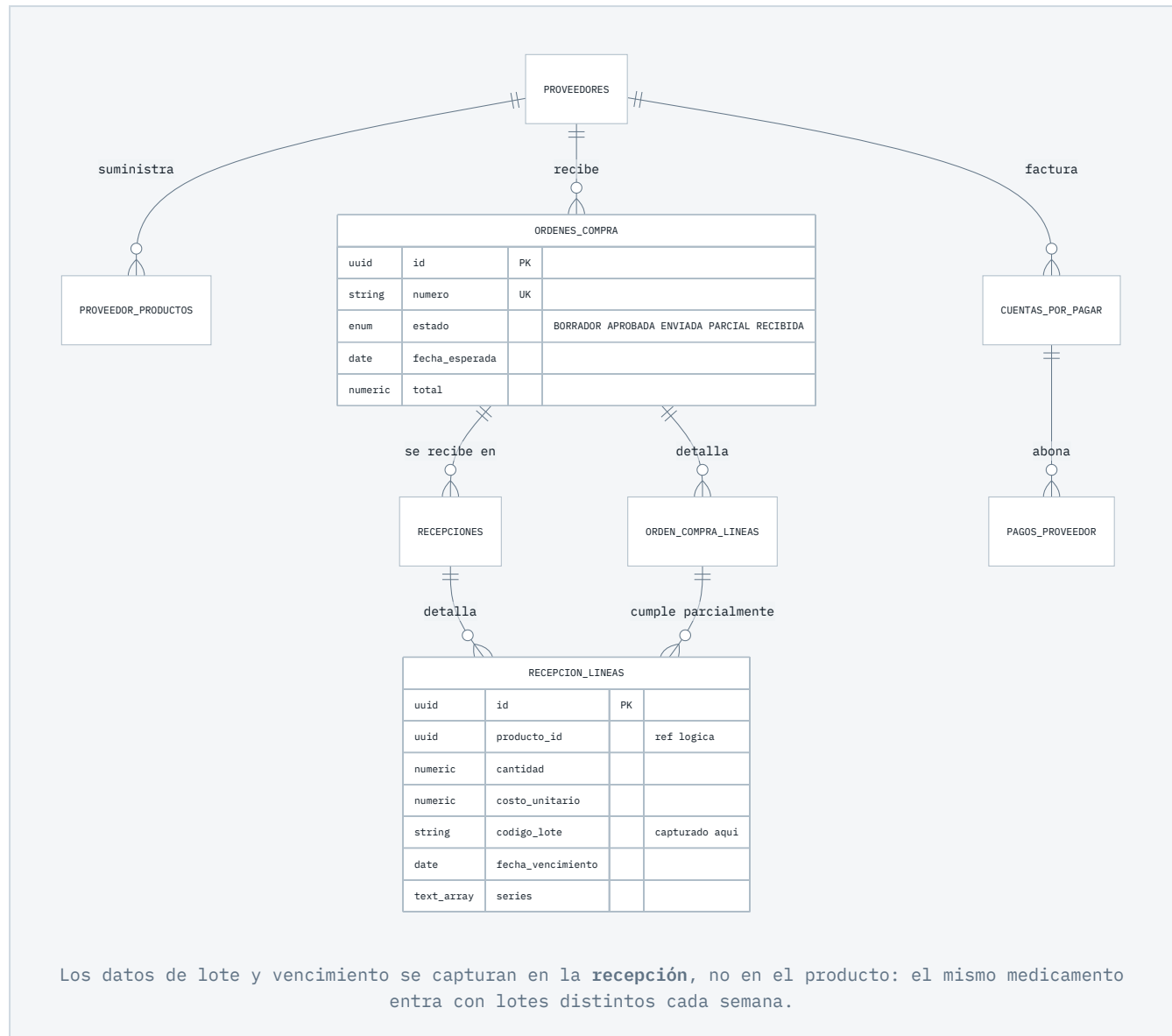
La saga Ventas ↔ Inventario

El documento lo menciona como riesgo asumido: «si Ventas descuenta stock e Inventario falla, ya no hay un `@Transactional` que cubra ambos». La máquina de estados queda persistida en `sagas` , y el stock apartado en `inventario.reservas_stock` con expiración.

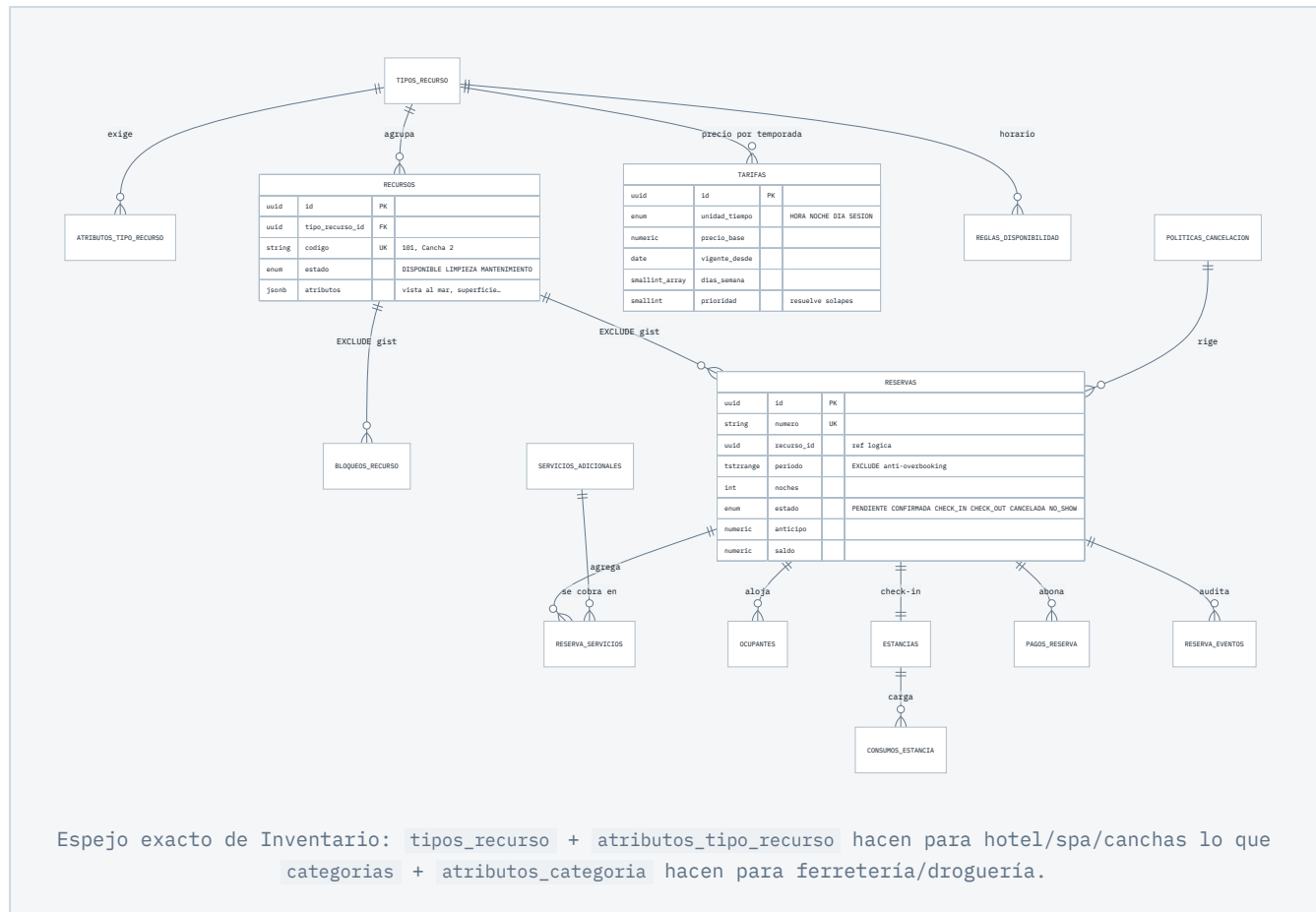


`reservas_stock.expira_en` cubre el caso en que Ventas muera entre 2 y 5: un job libera lo apartado y nada queda bloqueado para siempre.

Lo que llena el inventario. Sin este módulo el stock solo baja.



Un recurso no se consume, se **ocupa** durante un intervalo. Por eso no hay «stock» sino disponibilidad sobre una línea de tiempo, y por eso el periodo se modela como **TSTZRANGE** y no como dos columnas sueltas.



El punto técnico del patrón Reserva: el overbooking no se evita en Java

Un «¿está libre?» seguido de un **INSERT** tiene una ventana de carrera: dos peticiones concurrentes leen «libre» y ambas insertan. La garantía real es un constraint de exclusión GiST, que es atómico a nivel de base de datos.

```
CONSTRAINT ex_reserva_solape EXCLUDE USING gist (
    negocio_id WITH =,
    recurso_id WITH =,
    periodo WITH && -- && = "se solapa con"
) WHERE (recurso_id IS NOT NULL
    AND estado IN ('PENDIENTE', 'CONFIRMADA', 'CHECK_IN'))
```

VERIFICADO CONTRA POSTGRESQL 16

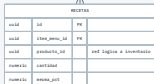
Solape prohibido — dos reservas CONFIRMADAS del mismo recurso, 10–13 sep y 12–15 sep → ERROR: conflicting key value violates exclusion constraint "ex_reserva_solape".

Cancelación libera — la misma pareja con la segunda en CANCELADA → las dos filas entran. El **WHERE** del constraint hace que cancelar libere el recurso sin borrar el histórico.

Check-out = check-in — 10–13 sep y 13–16 sep → ambas entran. El rango [inicio, fin) es semiabierto: la habitación se reutiliza el mismo día que se desocupa, que es exactamente como opera un hotel.

Consecuencia de diseño: `reservas.noches` **no** puede ser una columna **GENERATED**. El cast `timestampz → date` depende de la zona horaria de la sesión y Postgres exige expresiones inmutables. Lo calcula el servicio con la zona del

negocio y lo persiste.



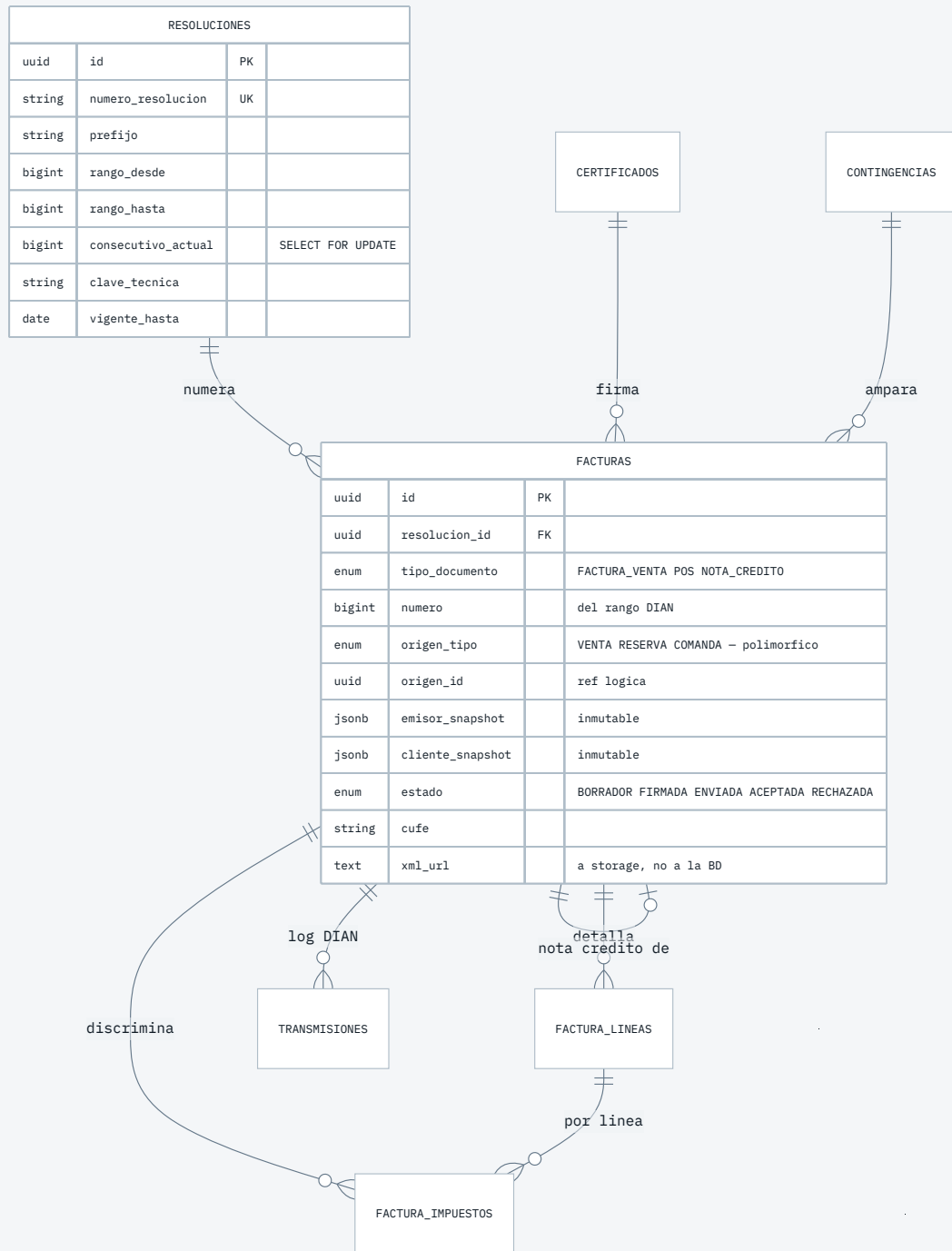
Cuatro cosas que este patrón necesita y los otros dos no: estado por línea, secuencia de envío por curso, división de cuenta y explosión de recetas.

HALLAZGO: LA PIEZA QUE FALTA EN EL DOCUMENTO ORIGINAL

Ni el PDF ni el `CLAUDE.md` dicen cómo se descuenta el inventario en un restaurante. Un plato no es un producto con stock: **consume insumos**. Sin la tabla `recetas` el patrón Comanda queda desconectado de Inventario y el dueño no puede saber cuánta carne le queda. Al cerrar una comanda se explota cada línea contra su receta y se publica `insumos_consumidos`, que Inventario consume para descontar. Ver hallazgo **04**.

Lo que hace distinta a una comanda de una venta

	VENTA	COMANDA
Duración	Un instante	90 minutos; se siguen agregando ítems después de enviar los primeros
Ciclo de vida	Solo la cabecera	Cabecera y cada línea : pendiente → enviada → en preparación → lista → entregada
Totales	Se calculan una vez al confirmar	Se recalculan en cada adición
Anular una línea	Se borra del borrador	Antes de enviar: se borra. Después de enviar: genera merma — el insumo ya se gastó. Son dos operaciones distintas.
Cobro	Uno o varios pagos, un total	La cuenta se divide entre comensales sin rehacer la comanda



Una factura emitida nunca se edita ni se borra: se anula con una nota crédito que la referencia.

DOS DECISIONES QUE SALEN DEL ANÁLISIS

Origen polimórfico. El documento dice «Facturación depende de Ventas». Con tres patrones eso ya no alcanza: también factura Reservas y Comandas. Por eso `origen_tipo + origen_id` en vez de una FK a `ventas`, y por eso consume tres eventos distintos.

El consecutivo no puede ser un `BIGSERIAL`. La DIAN autoriza un *rango* por resolución y exige numeración continua, sin huecos, dentro de él. Se toma con `SELECT ... FOR UPDATE` sobre `resoluciones.consecutivo_actual` dentro de la transacción, con un `CHECK` que impide salirse del rango. Ver hallazgo **05**.

caja — POS y arqueo

No puede vivir dentro de Ventas: en el patrón Comanda la caja también recibe pagos de comandas, y en Reserva recibe anticipos. `movimientos_caja` usa el mismo `origen_tipo/origen_id` polimórfico. `sesiones_caja` guarda `monto_esperado` (calculado) contra `monto_declarado` (contado) y expone `diferencia` como columna generada; `arqueo_denominaciones` guarda el conteo de billetes y monedas.

alertas — motor genérico, no reglas hardcodeadas

«Alertar vencimientos en droguería» no es código: es una fila en `reglas_alerta` con `tipo = 'VENCIMIENTO_LOTE'` y una `condicion` JSONB. `alertas.huella` tiene un `UNIQUE` que deduplica: la misma alerta no se repite dentro de la ventana de silencio. `dispositivos_push` guarda los tokens FCM por usuario y plataforma.

reportes — read model, no vistas sobre Ventas

Modelarlo como «vistas sobre las tablas de Ventas» rompería la regla de que ningún servicio accede a la base de otro y, con microservicios, ni siquiera sería posible. Es un esquema en estrella alimentado por eventos: `hechos_venta`, `hechos_reserva`, `hechos_comanda`, `hechos_inventario` (particionados por fecha) contra `dim_fecha`, `dim_producto`, `dim_cliente`, `dim_usuario`, `dim_sucursal`. Las dimensiones son **SCD tipo 2**: si cambia el nombre de un producto, el reporte histórico sigue mostrando el que tenía entonces. `agregados_diarios` y `ranking_productos` son lo que consume el dashboard de la app.

auditoria — dos problemas distintos que el documento junta

El PDF los agrupa como «Sincronización / auditoría»; aquí se separan porque no se parecen.

- **Auditoría** — `eventos_auditoria`, append-only, particionada por mes, con `cambios` JSONB de solo lo que cambió y un `trace_id` que correlaciona el request entre servicios.
- **Sincronización** — `operaciones_sync` es la cola que sube `workmanager`, con `idempotency_key` y `secuencia_local` para aplicar en el orden en que ocurrió en el dispositivo; `conflictos_sync` guarda versión de servidor contra versión de cliente y cómo se resolvió; `cambios_servidor` es el watermark de bajada.

El espejo local en Drift (cliente Flutter)

El cliente no replica las 147 tablas: guarda el subconjunto que necesita operar sin señal, más dos tablas propias — `cola_sincronizacion` (entidad, operación, payload, intentos, `idempotency_key`) y `meta_sync` (último cursor por entidad). En Android corre sobre SQLite nativo y en Web sobre `sqlite3.wasm`: mismo código Dart, motor distinto, y límites de almacenamiento distintos que conviene tener presentes al decidir cuánto catálogo cachear.

06 Modelo de dominio: agregados e invariantes

Qué se guarda junto, qué invariante protege el código y cuál protege la base de datos. La columna de la derecha es la que suele decidirse mal: una regla de concurrencia defendida solo en Java no está defendida.

AGREGADO RAÍZ	CONTIENE	INVARIANTE QUE PROTEGE EL CÓDIGO	INVARIANTE QUE PROTEGE POSTGRES
Negocio	Configuración, impuestos, sucursales	El patrón operativo no cambia si ya hay transacciones	UQ (pais, tipo_doc, numero)
Usuario	Roles asignados, sesiones	No exceder <code>plan.max_usuarios</code>	UQ (negocio_id, email)
Producto	Códigos, imágenes, precios por lista	<code>atributos</code> válido contra <code>atributos_categoria</code> ; <code>perecedero</code> ⇒ <code>maneja_lotes</code>	UQ sku · CHECK lotes · GIN
Existencia	Saldo por (producto, bodega)	No vender bajo cero salvo que el negocio lo permita	GENERATED disponible · CHECK
Venta	Líneas, pagos	Totales = suma de líneas; no se modifica tras confirmar	UQ numero · UQ origen_offline_id
OrdenCompra	Líneas, recepciones	No recibir más de lo pedido (tolerancia 5%)	CHECK cantidad_recibida
Reserva	Ocupantes, servicios, estancia, pagos	Política de cancelación y penalización	EXCLUDE gist – anti-overbooking
Comanda	Líneas, modificadores, cuentas, pagos	No cerrar con ítems sin enviar a cocina	UQ parcial: una abierta por sesión de mesa
SesionMesa	Mesas unidas	Cerrar libera la mesa a estado SUCIA	UQ parcial: una abierta por mesa
Factura	Líneas, impuestos, transmisiones	Emitida ⇒ inmutable; anular = nota crédito	CHECK rango · UQ (prefijo, numero)
SesionCaja	Movimientos, denominaciones, turnos	Cerrar exige arqueado declarado	UQ parcial: una abierta por caja
MovimientoInventario	—	Append-only; se corrige con un contrario	UQ idempotency_key

07 Mapa de eventos

Nombres en `snake_case` y participio pasado, como pide el `CLAUDE.md`. Los tres patrones publican su propio evento de cierre; los consumidores tratan a los tres igual.

EVENTO	PUBLICA	CONSUMEN	EFFECTO
venta_completada	ventas	facturacion · reportes · ctm · caja · inventario	Cierre del patrón VENTA DIRECTA
estancia_finalizada	reservas	facturacion · reportes · ctm · caja	Cierre del patrón RESERVA
pedido_completado	comandas	facturacion · reportes · ctm · caja · mesas	Cierre del patrón COMANDA
insumos_consumidos	comandas	inventario	Explosión de recetas → descuento stock
solicitar_reserva_stock	ventas	inventario	Paso 1 de la saga
stock_reservado	inventario	ventas	Paso 3a — confirma
stock_reserva_fallida	inventario	ventas	Paso 3b — compensa
stock_actualizado	inventario	reportes · menu · alertas	Marca ítems agotados en la carta
stock_bajo_minimo	inventario	alertas · compras	Alerta + sugerencia de reposición
lote_por_vencer	inventario	alertas	La «regla especial» de droguería, sin código nuevo
recepcion_registrada	compras	inventario	Da entrada y recalcula costo promedio
reserva_confirmada	reservas	alertas · reportes · ctm	Recordatorio de llegada
linea_lista	comandas	mesas · alertas	Avisa al mesero desde el KDS
factura_emitida / factura_rechazada	facturacion	ventas · reservas · comandas · alertas	Actualiza <code>estado_factura</code> en el origen
caja_descuadrada	caja	alertas · auditoria	Notifica al gerente
plan_cambiado	core_identidad	todos	Invalida la caché de módulos activos

08 Inventario completo de tablas

Las 147 tablas del modelo, por esquema. Las particiones (*_default) no se cuentan.

ESQUEMA	TABLAS
core_identidad 21	patrones_operativos · modulos · modulo_dependencias · planes · plan_modulos · permisos · plantillas_rol · plantilla_rol_permisos · negocios · suscripciones · negocio_modulos · configuracion_negocio · impuestos · sucursales · usuarios · roles · rol_permisos · usuario_rols · usuario_sucursales · refresh_tokens · invitaciones
crm 7	clientes · direcciones_cliente · contactos_cliente · interacciones · cliente_metricas · cuentas_por_cobrar · recaudos
inventario 21	categorias · atributos_categoria · unidades_medida · marcas · productos · producto_imagenes · producto_codigos · listas_precios · precios_producto · bodegas · existencias · lotes · existencias_lote · series · movimientos_inventario · traslados · traslado_lineas · ajustes_inventario · ajuste_lineas · reservas_stock · (+ partición)
ventas 8	consecutivos · ventas · venta_lineas · pagos_venta · devoluciones · devolucion_lineas · cotizaciones · sagas
compras 8	proveedores · proveedor_productos · ordenes_compra · orden_compra_lineas · recepciones · recepcion_lineas · cuentas_por_pagar · pagos_proveedor
recursos 8	tipos_recurso · atributos_tipo_recurso · recursos · tarifas · reglas_disponibilidad · bloqueos_recurso · servicios_adicionales · politicas_cancelacion
reservas 9	consecutivos · reservas · cupos_tipo_recurso · reserva_servicios · ocupantes · estancias · consumos_estancia · pagos_reserva · reserva_eventos
menu 10	cartas · categorias_menu · estaciones_cocina · items_menu · grupos_modificadores · modificadores · item_grupos_modificadores · recetas · combo_items · disponibilidad_diaria
mesas 5	zonas · mesas · sesiones_mesa · sesion_mesas · reservas_mesa
comandas 9	consecutivos · comandas · comanda_lineas · comanda_linea_modificadores · cuentas · cuenta_lineas · pagos_comanda · tickets_cocina · ticket_cocina_lineas
facturacion 7	resoluciones · facturas · factura_lineas · factura_impuestos · transmisiones · certificados · contingencias
caja 5	cajas · sesiones_caja · movimientos_caja · arqueo_denominaciones · turnos
alertas 6	tipos_alerta · reglas_alerta · alertas · dispositivos_push · entregas · preferencias_notificacion
reportes 16	dim_fecha · dim_producto · dim_cliente · dim_usuario · dim_sucursal · hechos_venta · hechos_reserva · hechos_comanda · hechos_inventario · agregados_diarios · ranking_productos · ocupacion_diaria · definiciones_reporte · reportes_programados · ejecuciones_reporte · posicion_consumo
auditoria 6	eventos_auditoria · dispositivos · operaciones_sync · conflictos_sync · cambios_servidor · politicas_retencion
public 2	outbox_eventos · inbox_eventos (se replican en cada base de servicio)

09 Catorce hallazgos del análisis

Puntos donde el documento de arquitectura y el `CLAUDE.md` están incompletos o donde el modelo de datos propuesto no aguanta lo que los planes prometen. Ordenados por severidad.

01 stock como columna de productos no aguanta multi-sucursal

ALTA

El modelo del documento define `productos(... precio, stock, atributos JSONB)`. Funciona con una bodega. El plan Empresarial vende Multi-sucursal y POS necesita saber de qué bodega salió la mercancía; además, sin un libro de movimientos no hay forma de auditar un descuadre ni de calcular rotación.

RESUELTO EN EL DDL

existencias(producto_id, bodega_id) como proyección + movimientos_inventario append-only y particionado como fuente de verdad. sucursal_id existe desde el día 1 aunque solo se venda en Empresarial: agregarla después obliga a reescribir todos los índices.

02 Cuatro módulos de la tabla de planes no tienen servicio en el scaffold

ALTA

El `CLAUDE.md` lista `gateway · usuarios · facturacion · reportes · inventario · ventas · compras · recursos · reservas · menu · mesas · comandas`. Pero la tabla de módulos y planes incluye **Cientes/CRM, Alertas, POS/Caja y Sincronización/auditoría**, que no aparecen. Se venden en un plan pero no tienen dónde vivir.

PROPUESTO

`servicio-clientes`, `servicio-alertas`, `servicio-caja` y `servicio-auditoria`. Ninguno puede alojarse dentro de Ventas: los cuatro son transversales a los tres patrones.

03 La tabla negocios no tiene servicio declarado

ALTA

Es la tabla raíz de todo el modelo multi-tenant y ningún servicio del scaffold la reclama. Si cada microservicio la consultara en cada request, el servicio dueño se vuelve un cuello de botella y un punto único de falla.

DECIDIDO

Vive en `servicio-usuarios` junto con planes, módulos y configuración. Los demás servicios

NO LA CONSULTAN

: reciben `negocio_id`, plan y patrón como claims del JWT y mantienen una caché invalidada por el evento `plan_cambiado`. Alternativa si el servicio crece: extraer un `servicio-tenant`.

04 El patrón Comanda no tiene forma de descontar inventario

ALTA

El documento equipara «Menú» a «Inventario», pero un ítem de menú no tiene stock: consume insumos. Sin ese puente, un restaurante en Regenta nunca sabe cuánta carne le queda, y el módulo de Compras queda inútil para el patrón Comanda.

RESUELTO EN EL DDL

Tabla `menu.recetas(item_menu_id, producto_id, cantidad, merma_pct)`. Al cerrar la comanda se explotan las líneas y se publica `insumos_consumidos`. También `modificadores.producto_id`, para que «extra queso» descuenta queso.

05 El consecutivo de facturación no puede ser un autoincremento

ALTA

La DIAN autoriza un *rango* por resolución y exige numeración continua, sin huecos, dentro de ese rango y dentro de su vigencia. Un `BIGSERIAL` deja huecos ante cualquier rollback y no conoce el rango.

RESUELTO EN EL DDL

`resoluciones.consecutivo_actual` tomado con `SELECT ... FOR UPDATE` dentro de la transacción, con `CHECK` de rango y un índice parcial que garantiza una sola resolución vigente por tipo y sucursal. Misma técnica en `ventas.consecutivos`.

06 Facturación acoplada a Ventas rompe con tres patrones

ALTA

«Facturación depende de Core, Ventas, Configuración, Clientes». Un hotel y un restaurante también facturan, y su transacción no es una venta.

RESUELTO EN EL DDL

`facturas.origen_tipo + origen_id` polimórfico, y el servicio consume `venta_completada`, `estancia_finalizada` y `pedido_completado`. La dependencia declarada correcta es «el módulo de transacción del patrón activo», no «Ventas».

07 El overbooking no se puede evitar validando en el servicio

ALTA

Es el riesgo más caro del patrón Reserva y no está mencionado. Un «¿está libre?» seguido de un `INSERT` tiene una ventana de carrera; dos recepcionistas vendiendo la misma habitación al mismo tiempo pasan ambos.

RESUELTO Y VERIFICADO

`EXCLUDE USING gist (negocio_id =, recurso_id =, periodo &&)` con `WHERE estado IN (...)`. Requiere `periodo` como `TSTZRANGE` y la extensión `btree_gist`. Probado contra PostgreSQL 16: rechaza el solape, permite el solape con una cancelada, y permite check-in el mismo día del check-out.

08 Offline-first con claves secuenciales es imposible

ALTA

El documento promete «el vendedor registra una venta sin señal y se sincroniza después», pero no dice qué implica en el modelo. Con PK secuenciales el cliente no puede crear la venta ni referenciarla localmente, y un reintento de `workmanager` la duplica.

RESUELTO EN EL DDL

UUID (v7) generado en el cliente + `origen_offline_id` con `UNIQUE` por negocio + `idempotency_key` en todo lo que se aplique más de una vez + `version` para bloqueo optimista + `auditoria.operaciones_sync` y `conflictos_sync` para la cola y la resolución.

09 RLS necesita `SET LOCAL`, y con HikariCP eso es fácil de hacer mal

ALTA

El documento propone Row-Level Security «como respaldo», pero no menciona que la política depende de una variable de sesión. Un `SET` (sin `LOCAL`) deja el `negocio_id` pegado a la conexión del pool y el siguiente request de otro tenant ve datos ajenos: el mecanismo de seguridad se convierte en la fuga.

RESUELTO

Función `app_negocio_actual()` + `SET LOCAL app.negocio_id` en un interceptor AOP atado a la transacción (`TenantFilterAspect`), nunca en la configuración del pool. Y `FORCE ROW LEVEL SECURITY`, para que la política aplique también al owner de la tabla.

10 Sin Outbox, el evento se pierde y nadie se entera

MEDIA

El documento describe la comunicación asíncrona pero no cómo se garantiza. Publicar después del commit significa que un RabbitMQ caído hace desaparecer el evento: la venta existe y la factura nunca se emite, sin error visible en ningún lado.

RESUELTO EN EL DDL

`outbox_eventos` en cada base de servicio (INSERT en la misma transacción del agregado, publicador aparte) + `inbox_eventos` con `mensaje_id` como PK para la idempotencia de consumo, porque AMQP entrega *at-least-once*.

11 Reportes no puede leer las tablas de los otros servicios

MEDIA

«Reportes depende de Core + los módulos activos que reporte» sugiere consultas o JOINS sobre Ventas e Inventario. Con base por servicio eso no es una mala práctica: es imposible.

RESUELTO EN EL DDL

Esquema en estrella alimentado por eventos: tablas de hechos particionadas + dimensiones SCD tipo 2 replicadas. Un cambio de nombre de producto no reescribe el histórico. `posicion_consumo` permite reconstruir el read model desde cero.

12 Precios e impuestos leídos del catálogo corrompen el histórico

MEDIA

No está mencionado, y es de los errores más comunes: si la línea de venta solo guarda `producto_id`, subir el precio mañana cambia el total de las ventas de ayer, el margen de los reportes y el contenido de una factura ya emitida.

RESUELTO EN EL DDL

Snapshots en toda línea de documento (`sku_snapshot`, `nombre_snapshot`, `precio_unitario`, `impuesto_pct`, `costo_unitario_snapshot`) y `cliente_snapshot` / `emisor_snapshot` JSONB en `facturas`.

13 Multi-sucursal se vende en Empresarial pero afecta el modelo desde el inicio

MEDIA

El documento lo advierte («conviene pensarlo desde el diseño aunque no se implemente de una») y tiene razón: cambia inventario, ventas, caja, mesas y recursos a la vez.

RESUELTO EN EL DDL

`sucursal_id` nullable en todas las tablas afectadas desde el principio, con `NULL` = negocio de un solo punto. Habilitar el módulo es poblar `sucursales`, no migrar el esquema.

14 Tres columnas obvias no pueden ser GENERATED: el cast a fecha no es inmutable

BAJA

Detectado al ejecutar el DDL contra PostgreSQL 16, no en el papel. `reservas.noches` como `upper(periode)::date - lower(periode)::date` es rechazado, y lo mismo un índice sobre `lower(periode)::date`: el cast `timestampz → date` depende de la zona horaria de la sesión.

RESUELTO EN EL DDL

`noches` se calcula en el servicio con la zona horaria del negocio y se persiste; el índice de llegadas va sobre `lower(periode)` como `TIMESTAMPTZ` y el servicio consulta con un rango construido en la zona del negocio. Mismo motivo por el que `negocios.zona_horaria` es obligatoria.

10 Archivos entregados

En `C:\logula\APP_CLAUDE\Regenta\modelo-datos\`. Los dieciséis scripts SQL se ejecutaron completos contra PostgreSQL 16 sin un solo error, en el orden numérico.

<code>sql/00-convenciones.sql</code>	Extensiones, tipos, RLS, Outbox/Inbox, idempotencia
<code>sql/01-servicio-usuarios.sql</code>	Tenant, planes, módulos, identidad, roles, sucursales
<code>sql/02-servicio-clientes.sql</code>	CRM y cartera
<code>sql/03-servicio-inventario.sql</code>	Catálogo configurable, stock, lotes, libro mayor
<code>sql/04-servicio-ventas.sql</code>	Transacción, pagos, devoluciones, saga
<code>sql/05-servicio-compras.sql</code>	Proveedores, órdenes, recepciones, CxP
<code>sql/06-servicio-recursos.sql</code>	Tipos, recursos, tarifas, bloqueos
<code>sql/07-servicio-reservas.sql</code>	Reservas con EXCLUDE, estancias, consumos
<code>sql/08-servicio-menu.sql</code>	Cartas, ítems, modificadores, recetas
<code>sql/09-servicio-mesas.sql</code>	Zonas, mesas, sesiones
<code>sql/10-servicio-comandas.sql</code>	Comandas, líneas con estado, cuentas, KDS
<code>sql/11-servicio-facturacion.sql</code>	Resoluciones DIAN, facturas, transmisiones
<code>sql/12-servicio-caja.sql</code>	Cajas, arqueos, movimientos
<code>sql/13-servicio-alertas.sql</code>	Motor de reglas, entregas, tokens FCM
<code>sql/14-servicio-reportes.sql</code>	Estrella: hechos, dimensiones, agregados
<code>sql/15-servicio-auditoria.sql</code>	Bitácora, cola de sync, conflictos
<code>jpa/</code>	Entidades Java: base multi-tenant, @RequiereModulo, Outbox y los 6 agregados raíz

Cómo levantarlo en local

```
# Con el docker compose del CLAUDE.md, o un Postgres 16 cualquiera
createdb regenta
psql -d regenta -f sql/00-convenciones.sql
for f in sql/0[1-9]*.sql sql/1*.sql; do psql -d regenta -f "$f"; done
```

Todos los esquemas caben en una sola base para revisar el modelo entero de un vistazo. **En producción cada esquema va a su propia base**, una por microservicio, como manda el `CLAUDE.md`: los scripts no tienen ni una FK que cruce de un esquema a otro, así que separarlos no rompe nada.

Por dónde seguir

El orden que respeta las dependencias, un módulo por sesión como pide el flujo de trabajo del `CLAUDE.md`: `core_identidad` → `inventario` → `ventas` (con la saga) → `crm` → `facturacion`. Recién ahí el segundo patrón. Y antes de generar código de un módulo, actualizar el `CLAUDE.md` con los cuatro servicios propuestos en el hallazgo **02** — si no están descritos ahí, el primer paso del flujo de trabajo falla.